

EvalFlow

Modern Examination Evaluation Platform — Project Report

1. Problem Statement & Objectives

Problem: Traditional online examination systems rely on fragmented workflows — students submit answers via email, evaluators receive scripts informally, and payments happen outside the system. This leads to lost scripts, delayed grading, zero transparency in marking, and no financial audit trail.

Objective: Build a centralized platform connecting **Administrators**, **Students**, and **Evaluators**. Automate: course creation → question paper design → timed submissions → per-question grading → financial ledger with SSLCommerz payments.

2. Executive Summary

EvalFlow is a full-stack web application (Laravel 11 + MySQL) with three role-based dashboards:

- Administrator** — Creates courses/exams/questions, manages users, assigns evaluators, oversees revenue and evaluator payouts.
- Student** — Deposits credit via SSLCommerz (bKash/Nagad/cards), accesses time-locked papers, uploads PDF scripts, views per-question results.
- Evaluator** — Grades scripts via split-screen PDF viewer, earns ₳0.75/mark, withdraws to mobile wallets.

3. Technology Stack

Layer	Technology	Why
Backend	Laravel 11 (PHP 8.3)	MVC with ORM, routing, middleware, file storage
Auth	Laravel Breeze	Login, registration, password reset scaffolding
Database	MySQL 8.x	Foreign keys, ENUMs, cascading constraints
ORM	Eloquent	Model relationships, query building, mass-assignment protection
Frontend	Blade + CSS	Server-rendered views with custom layout
PDF Viewer	<iframe>	Browser's built-in PDF renderer, zero dependencies
Payment	SSLCommerz Sandbox	Bangladesh gateway: bKash, Nagad, cards with IPN
Timezone	Asia/Dhaka (GMT+6)	All timestamps in Bangladesh Standard Time

4. Middleware Logic — `app/Http/Middleware/`

File	Alias	Logic
<code>RoleMiddleware.php</code>	<code>role:{role}</code>	Checks <code>\$request->user()->role !== \$role</code> → <code>abort(403)</code>
<code>EnsureExamTimeIsValid.php</code>	<code>exam.time</code>	Rejects if <code>strtotime(\$end) <= strtotime(\$start)</code>
<code>CheckPasswordStrength.php</code>	global	Validates password against <code>Password::defaults()</code> rules
<code>EnsureExamHasQuestions.php</code>	<code>exam.questions</code>	Redirects if <code>\$exam->questions()->count() === 0</code>
<code>EnsureMinimumWithdrawal.php</code>	<code>withdraw.min</code>	Rejects if <code>\$request->amount < 100</code>

5. Database Schema

8 core tables with foreign key constraints, cascading deletes, and ENUM status fields.

5.1 `users` — `migrations/0001_01_01_000000_create_users_table.php`

All system users. `role` determines dashboard redirect. `balance` acts as digital wallet — students pay exam fees, evaluators accumulate earnings.

Column	Type	Constraints	Purpose
<code>id</code>	BIGINT	PK, AUTO_INCREMENT	Unique ID
<code>name</code>	VARCHAR	NOT NULL	Display name
<code>email</code>	VARCHAR	UNIQUE	Login email
<code>role</code>	VARCHAR	DEFAULT 'student'	admin / student / evaluator
<code>balance</code>	DECIMAL(10,2)	DEFAULT 0.00	Wallet (₹)
<code>password</code>	VARCHAR	NOT NULL	Bcrypt hash
<code>remember_token</code>	VARCHAR	NULLABLE	Session token
<code>created_at</code> / <code>updated_at</code>	TIMESTAMP	NULLABLE	Timestamps

5.2 courses — migrations/2026_07_01_035124_create_courses_table.php

Groups exams under academic courses. Unique code prevents duplicates.

Column	Type	Constraints	Purpose
id	BIGINT	PK	Unique ID
name	VARCHAR	NOT NULL	Course name
code	VARCHAR	UNIQUE	Code (e.g. CSE3110)
created_at / updated_at	TIMESTAMP	NULLABLE	Timestamps

5.3 exams — migrations/2026_07_01_035125_create_exams_table.php

Central table. `start_time` / `end_time` define when students can access papers and submit. `total_marks` is auto-computed from questions via Eloquent accessor. One evaluator per exam.

Column	Type	Constraints	Purpose
id	BIGINT	PK	Unique ID
course_id	BIGINT	FK → courses CASCADE	Parent course
title	VARCHAR	NOT NULL	Exam title
total_marks	INT	DEFAULT 100	Auto-computed
start_time / end_time	DATETIME	NOT NULL	Exam window
assigned_evaluator_id	BIGINT	FK → users NULL ON DELETE	Assigned grader
created_at / updated_at	TIMESTAMP	NULLABLE	Timestamps

5.4 questions — migrations/2026_07_02_050301_create_questions_table.php

Individual questions with mark weight. `media_path` allows PDF/image attachments. `order` controls display sequence.

Column	Type	Constraints	Purpose
<code>id</code>	BIGINT	PK	Unique ID
<code>exam_id</code>	BIGINT	FK → exams CASCADE	Parent exam
<code>body</code>	TEXT	NOT NULL	Question text
<code>marks</code>	INT UNSIGNED	DEFAULT 1	Max marks
<code>media_path</code>	VARCHAR	NULLABLE	PDF attachment
<code>order</code>	INT UNSIGNED	DEFAULT 0	Display order

5.5 scripts — migrations/2026_07_01_035126_create_scripts_table.php

Each student's uploaded PDF answer. One submission per student per exam (controller check). Status starts `pending` → changes to `evaluated` when graded. `marks_obtained` = sum of per-question marks.

Column	Type	Constraints	Purpose
<code>id</code>	BIGINT	PK	Unique ID
<code>exam_id</code>	BIGINT	FK → exams CASCADE	Which exam
<code>student_id</code>	BIGINT	FK → users CASCADE	Who submitted
<code>file_path</code>	VARCHAR	NOT NULL	PDF storage path
<code>status</code>	ENUM	pending / evaluated	Grading status
<code>marks_obtained</code>	INT	NULLABLE	Total after grading

5.6 `script_marks` — `migrations/2026_07_04_052834_create_script_marks_table.php`

Per-question grading. Each row = marks for one question on one script. `UNIQUE(script_id, question_id)` prevents duplicates. Enables the detailed breakdown students see in results.

Column	Type	Constraints	Purpose
<code>id</code>	BIGINT	PK	Unique ID
<code>script_id</code>	BIGINT	FK → scripts CASCADE	Which script
<code>question_id</code>	BIGINT	FK → questions CASCADE	Which question
<code>marks_obtained</code>	SMALLINT	NULLABLE	Marks awarded

5.7 `transactions` — `migrations/2026_07_10_061920_create_transactions_table.php`

All monetary movement. Four types: `deposit`, `exam_fee`, `earning`, `withdrawal`. `trx_id` links to SSLCommerz for verification.

Column	Type	Constraints	Purpose
<code>id</code>	BIGINT	PK	Unique ID
<code>user_id</code>	BIGINT	FK → users CASCADE	Owner
<code>type</code>	ENUM	<code>deposit/exam_fee/earning/withdrawal</code>	Category
<code>amount</code>	DECIMAL(10,2)	NOT NULL	Amount (₹)
<code>trx_id</code>	VARCHAR	UNIQUE, NULLABLE	SSLCommerz ID
<code>status</code>	ENUM	<code>pending/completed/failed</code>	Payment status

5.8 `withdrawals` — `migrations/2026_07_10_061921_create_withdrawals_table.php`

Evaluator payout requests. Balance deducted immediately (prevents double-spending). Admin approves/rejects. Min ₹100 enforced by `EnsureMinimumWithdrawal.php`.

Column	Type	Constraints	Purpose
<code>id</code>	BIGINT	PK	Unique ID
<code>user_id</code>	BIGINT	FK → users CASCADE	Evaluator
<code>amount</code>	DECIMAL(10,2)	NOT NULL	Amount (min ₹100)
<code>method</code>	ENUM	bkash/nagad/dbbl	Payout method
<code>account_number</code>	VARCHAR	NOT NULL	Recipient
<code>status</code>	ENUM	pending/approved/rejected	Admin status

6. Welcome Page & Authentication

6.1 Landing Page — `resources/views/welcome.blade.php`

Student-focused marketing page. Authenticated users see "Go to Dashboard" which triggers the role-based redirect in `routes/web.php`.

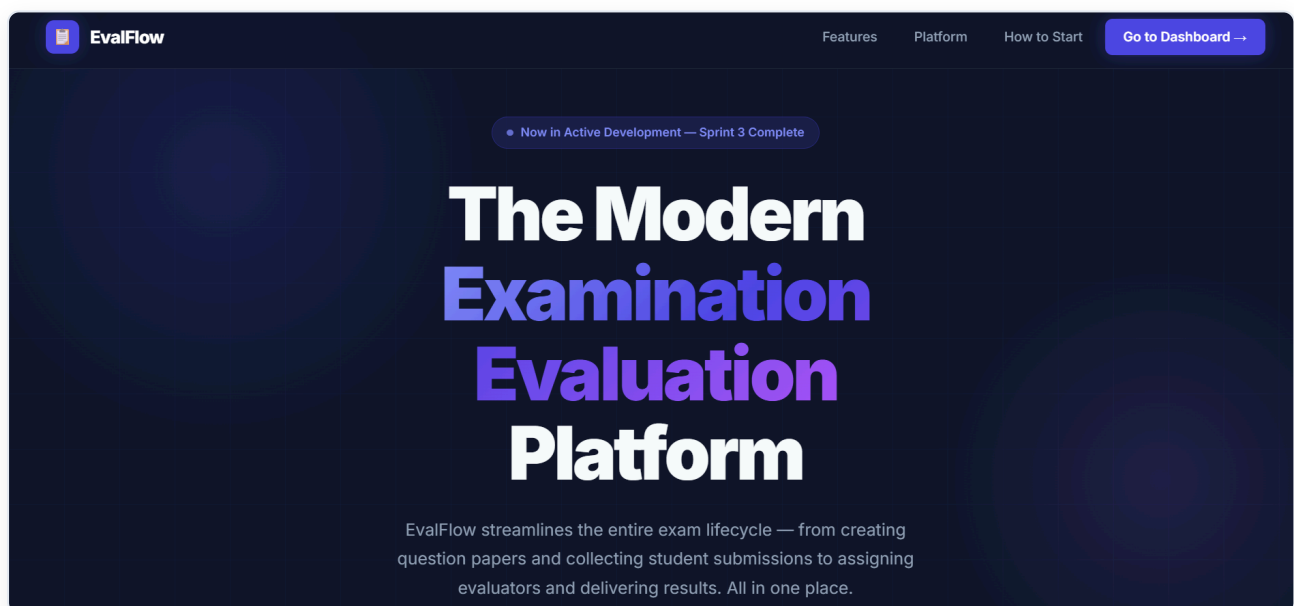


Figure 2: Landing page with feature highlights and CTAs.

7. Entity-Relationship (ER) Diagram

Exported from phpMyAdmin Designer. Shows all 8 entities with PK/FK constraints and relationship cardinalities.

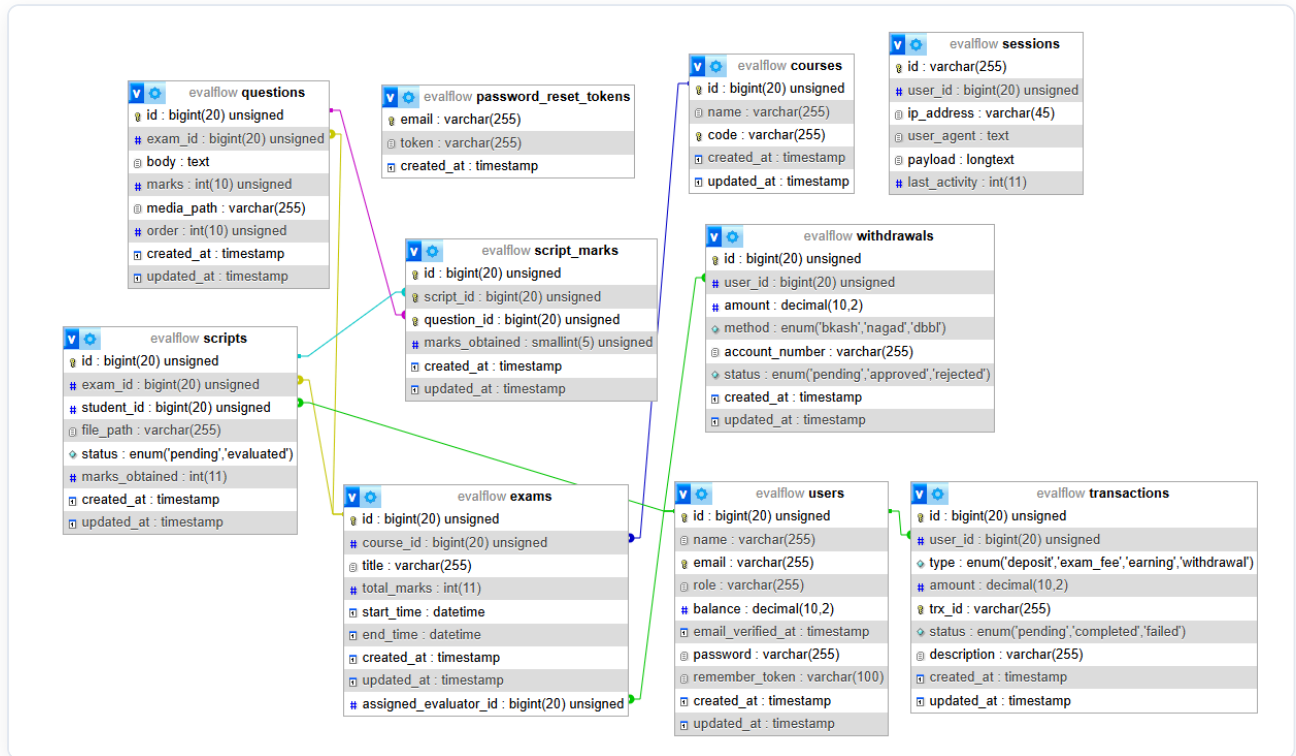


Figure 1: EvalFlow ER Diagram — 8 tables, 9 foreign key relationships.

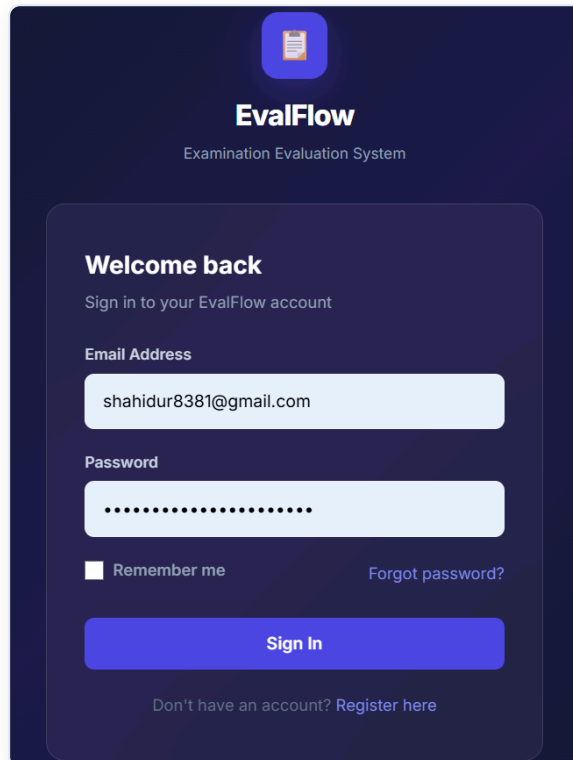
Relationship Summary

From → To	Type	FK Column	On Delete
courses → exams	1:M	exams.course_id	CASCADE
users → exams	1:M	exams.assigned_evaluator_id	SET NULL
exams → questions	1:M	questions.exam_id	CASCADE
exams → scripts	1:M	scripts.exam_id	CASCADE
users → scripts	1:M	scripts.student_id	CASCADE
scripts → script_marks	1:M	script_marks.script_id	CASCADE
questions → script_marks	1:M	script_marks.question_id	CASCADE

6.2 Login & Role Redirect

Login handled by `auth/login.blade.php`. After auth, `/dashboard` reads `user.role` and redirects:

```
// routes/web.php (line 10-15)
Route::get('/dashboard', function () {
    $role = auth()->user()->role;
    if ($role === 'admin') return redirect()->route('admin.dashboard');
    if ($role === 'evaluator') return redirect()->route('evaluator.dashboard');
    return redirect()->route('student.dashboard');
})->middleware(['auth', 'verified']);
```



EvalFlow
Examination Evaluation System

Welcome back
Sign in to your EvalFlow account

Email Address
shahidur8381@gmail.com

Password
.....

☐ Remember me [Forgot password?](#)

Sign In

Don't have an account? [Register here](#)

Figure 3: Login page.

8. Admin Panel

8.1 Route Protection

All admin routes are protected by two middleware layers in `routes/web.php` :

```
// routes/web.php (line 18-19)
Route::middleware(['auth', 'verified', 'role:admin'])
    ->prefix('admin')->name('admin.')->group(function () { ... });
```

If a student tries `/admin/dashboard` : `RoleMiddleware.php` checks `$request->user()->role !== 'admin'` → `abort(403)` .

8.2 Dashboard — `AdminExamController.php`

Handles course CRUD, exam CRUD, question management, and evaluator assignment.

- **Exam creation:** Passes through `EnsureExamTimeIsValid.php` middleware
- **Total marks:** Computed via `$exam->questions()->sum('marks')` in `Exam.php`
- **Evaluator assignment:** Sets `assigned_evaluator_id`
- **Cascade delete:** Removes questions, scripts, and script_marks

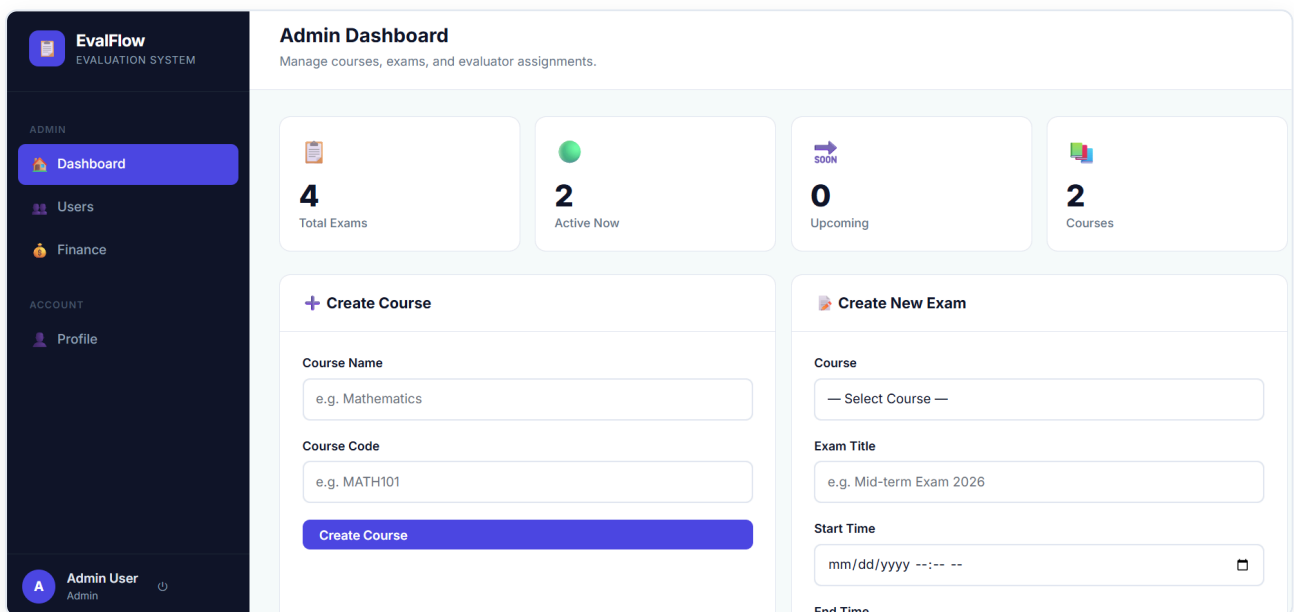


Figure 4: Admin dashboard — courses, exams, questions.

8.3 User Management — AdminUserController.php

View: `admin/users.blade.php`. Filter by role, search by name/email, create evaluator accounts, delete users with cascade (removes their transactions and scripts).

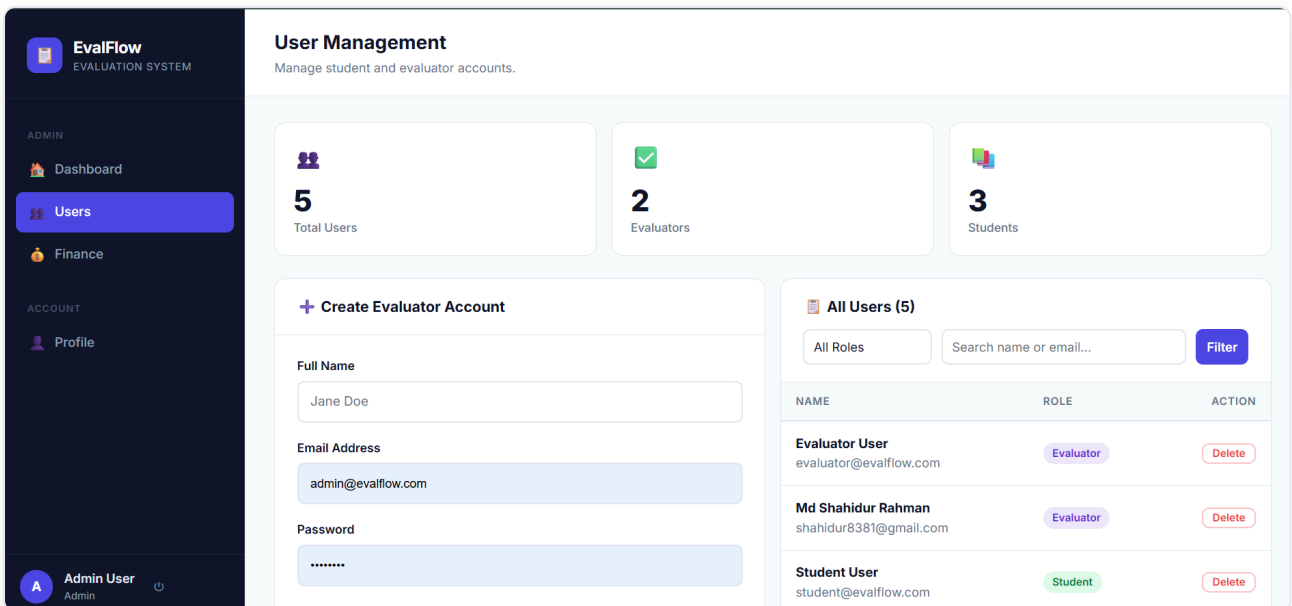


Figure 5: User management with role filter and evaluator creation.

8.4 Finance — AdminFinanceController.php

View: `admin/finance/index.blade.php`

- **Revenue tracking:** Aggregates `transactions` by type.
- **Payout approval:** Lists `withdrawals` with status `pending`. Admin calls `approveWithdrawal()` → sets status to `approved`.

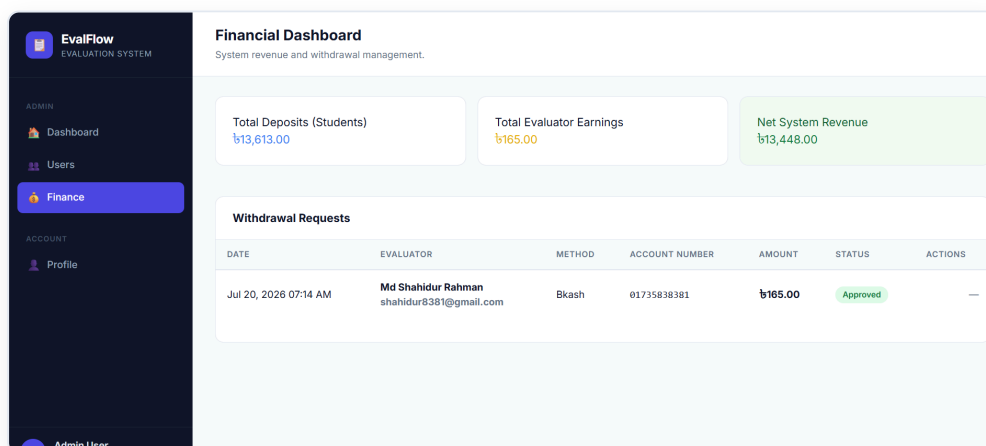


Figure 6: Revenue overview and pending evaluator payouts.

9. Student Panel

9.1 Exam List — `StudentExamController.php`

View: `student/dashboard.blade.php` . Categorizes exams by time:

```
$active = Exam::where('start_time', '<=', now())
    ->where('end_time', '>=', now())->get();

// Balance check in view:
@if(auth()->user()->balance < $exam->total_marks)
    Needs to {{ $exam->total_marks }}
@endif
```

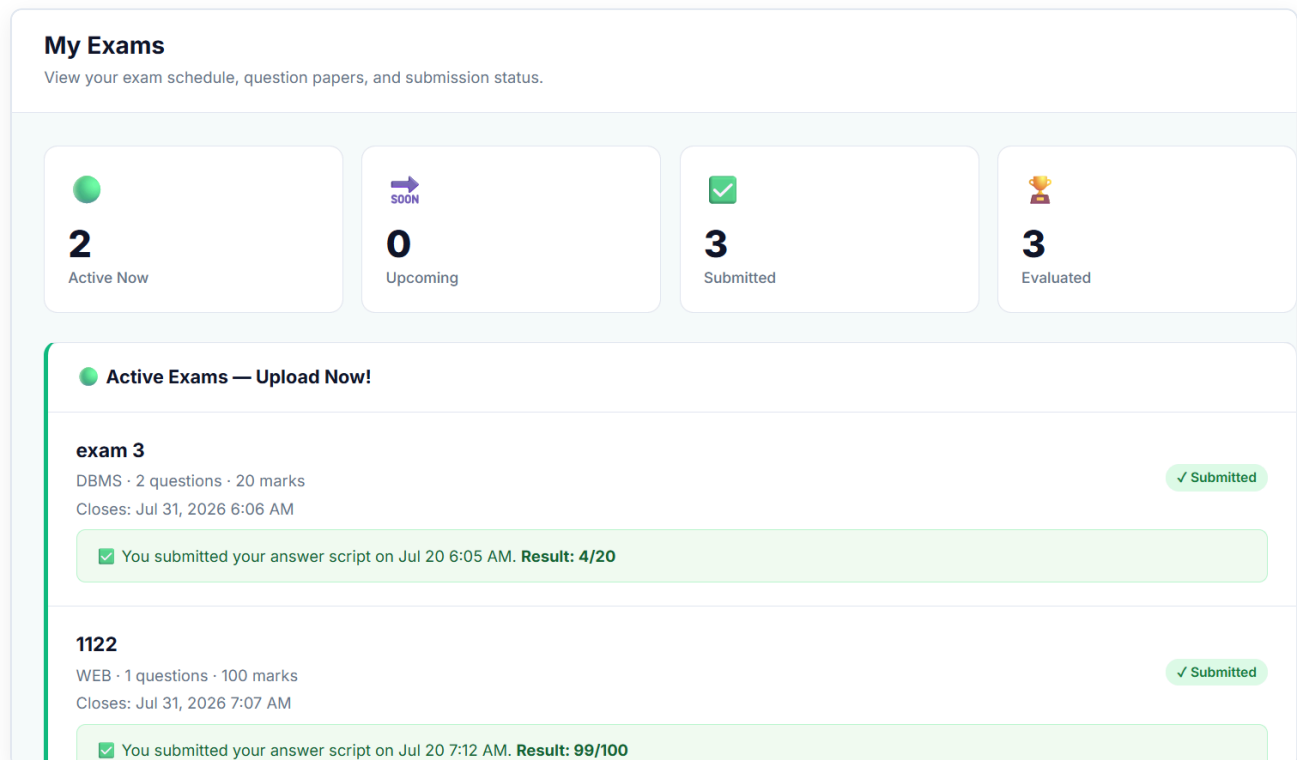


Figure 7: Student dashboard with Active / Upcoming / Past exams.

9.2 Exam View & Upload — `student/exam.blade.php`

Route passes through `EnsureExamHasQuestions.php` . Upload logic in `uploadScript()` :

```
// 1. Validate PDF file (max 20MB)
// 2. Check: now() is within [start_time, end_time]
// 3. Check: student hasn't already submitted, balance >= total_marks
// 4. Store PDF → storage/app/public/scripts/
// 5. Create Script (status: 'pending') & Deduct exam fee
// 6. Create Transaction (type: 'exam_fee')
```

1122
← Back

WEB · 1 questions · 100 marks total

This exam is **currently active** . You can upload your answer script below before **Jul 31, 2026 7:07 AM** .

Question Paper
100 marks

Question 1
100 marks

DFFDD

Upload Answer Script

Instructions:

- Write your answers on paper.
- Click photos with a PDF generator app (e.g. CamScanner) and upload the PDF.
- PDF files only (max 20MB).
- You can only submit **once** per exam.
- Make sure your script is complete before uploading.

Select PDF File

Choose File
No file chosen

Submit Answer Script

Figure 8: Question paper with marks and PDF upload form.

9.3 Wallet & SSLCommerz — StudentFinanceController.php

View: `student/finance/index.blade.php`

```
// 1. Student submits amount → initiatePayment()
$trx_id = uniqid('EVAL_');
Transaction::create([...status => 'pending', trx_id => $trx_id]);

// 2. POST to SSLCommerz session API
$response = Http::post('.../gwprocess/v4/api.php', [ ... ]);
return redirect($response['GatewayPageURL']);

// 3. IPN callback (OUTSIDE auth middleware) → success()
//   Validates trx_id, updates transaction → 'completed'
//   Increments user balance
```

Why outside auth? SSLCommerz callbacks are cross-origin POSTs. `SameSite=Lax` drops session cookies → auth middleware would reject. Callback routes in `routes/web.php` are outside the auth group.

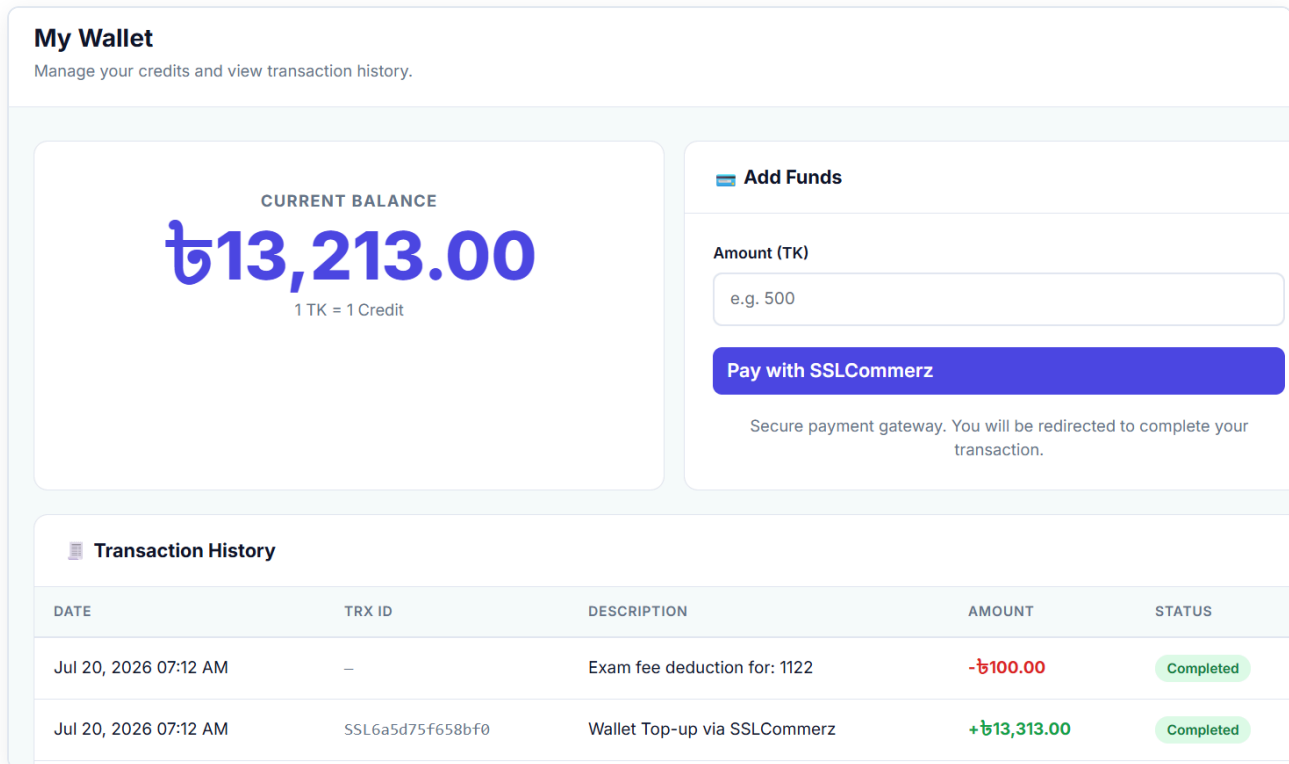


Figure 9: Wallet balance, SSLCommerz deposit, and transaction history.

10. Evaluator Panel

10.1 Dashboard & Grading — `EvaluatorScriptController.php`

Only shows scripts from assigned exams:

```
$exams = Exam::where('assigned_evaluator_id', auth()->id()->get());
```

Grading saves marks atomically using DB transactions:

```
DB::transaction(function () use ($script, $marks) {
    foreach ($marks as $questionId => $value) {
        ScriptMark::updateOrCreate(
            ['script_id' => $script->id, 'question_id' => $questionId],
            ['marks_obtained' => $value]
        );
    }
    $script->update([
        'marks_obtained' => $script->scriptMarks()->sum('marks_obtained'),
        'status' => 'evaluated'
    ]);
    // Credit evaluator: total_marks × 0.75
    $earning = $script->exam->total_marks * 0.75;
    auth()->user()->increment('balance', $earning);
    Transaction::create([...type => 'earning', amount => $earning]);
});
```

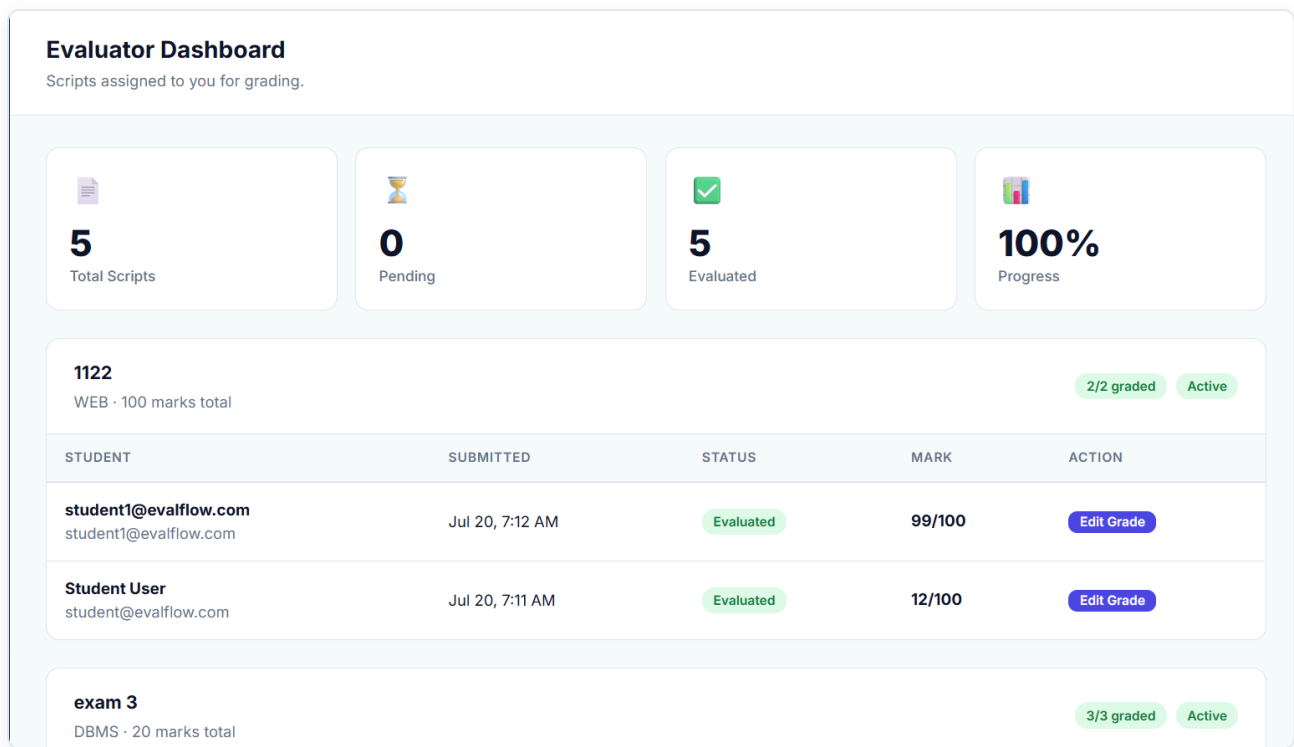


Figure 10: Evaluator dashboard and split-screen grading.

10.2 Withdrawal — EvaluatorFinanceController.php

Route passes through EnsureMinimumWithdrawal.php (alias `withdraw.min`):

```
// 1. Validate: amount <= auth()->user()->balance
// 2. Deduct balance IMMEDIATELY (prevents double-spending)
// 3. Create Withdrawal (status: 'pending')
// 4. Admin reviews → approves/rejects on admin finance page
```

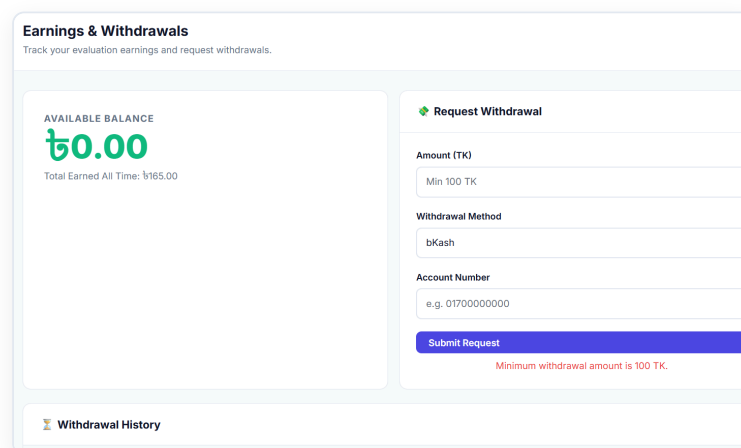


Figure 11: Evaluator finance — balance, withdrawal form, earnings history.

11. System Flow — Exam Lifecycle

1

Admin Creates Exam

`AdminExamController::storeExam()`

Route: `POST /admin/exams` → middleware: `auth → role:admin → exam.time`

2

Admin Adds Questions & Assigns Evaluator

`AdminExamController::addQuestion()` — `POST /admin/exams/{exam}/questions`

Sets `exams.assigned_evaluator_id`. Total marks auto-computed by `Exam.php::getTotalMarksAttribute()`.

3

Student Views Paper & Uploads Script

`GET /student/exams/{exam}` → middleware: `auth → role:student → exam.questions`

`StudentExamController::uploadScript()` — validates, stores PDF, deducts fee, creates Transaction.

4

Evaluator Grades Script

`POST /evaluator/scripts/{script}/grade` → middleware: `auth → role:evaluator`

`EvaluatorScriptController::storeMarks()` — DB transaction: saves per-question marks via

`ScriptMark::updateOrCreate()`, computes total, credits evaluator $0.75 \times \text{marks}$.

5

Student Views Results

`StudentExamController::showExam()` loads `$script->scriptMarks` with question data.

`student/exam.blade.php` renders per-question breakdown.

Exam Status State Machine

State	Condition	Student Action
Upcoming	<code>now() < start_time</code>	See metadata only
Active	<code>start_time ≤ now() ≤ end_time</code>	View questions, upload PDF
Submitted	Script exists, <code>status = 'pending'</code>	Wait for evaluation
Evaluated	<code>status = 'evaluated'</code>	View score + breakdown
Missed	<code>now() > end_time</code> , no script	Cannot submit

12. System Flow — Financial Flow

12.1 Student Deposit

1

POST /student/finance/deposit → `StudentFinanceController::initiatePayment()`

Creates `Transaction(status: pending)`. POSTs to SSLCommerz API. Redirects to gateway.

2

POST /student/finance/success → `StudentFinanceController::success()`

Outside auth middleware. Validates `trx_id` against DB. Updates status → `completed`.

12.2 Exam Fee Deduction

→

`StudentExamController::uploadScript()`

`auth()->user()->decrement('balance', $exam->total_marks)`

Creates `Transaction(type: exam_fee, status: completed)`

12.3 Evaluator Earning

→

`EvaluatorScriptController::storeMarks()`

`auth()->user()->increment('balance', $exam->total_marks * 0.75)`

Creates `Transaction(type: earning, status: completed)`

12.4 Evaluator Withdrawal

1

POST /evaluator/finance/withdraw → `EvaluatorFinanceController::withdraw()`

Validates `amount ≤ balance`. Deducts balance immediately. Creates `Withdrawal(status: pending)`.

2

POST /admin/finance/withdrawals/{id}/approve → `AdminFinanceController::approveWithdrawal()`

Sets `withdrawal.status = 'approved'`. Funds released externally by admin.

13. Authentication & Access Control

13.1 Request Pipeline (example: student accessing exam)

```
GET /student/exams/5

├─ 1. auth middleware (Laravel Breeze)
│   └─ Not logged in? Redirect to /login
├─ 2. role:student middleware (RoleMiddleware.php)
│   └─ $request->user()->role !== 'student'? abort(403)
├─ 3. exam.questions middleware (EnsureExamHasQuestions.php)
│   └─ $exam->questions()->count() === 0? Redirect with error
├─ 4. StudentExamController::showExam($exam)
│   └─ Balance check: auth()->user()->balance < $exam->total_marks?
│       └─ Disable upload button in view
│       └─ Time check: now() < start_time?
│           └─ Hide question paper
└─ 5. Return view: student/exam.blade.php
```

13.2 Route Architecture — routes/web.php

Group	Middleware Stack	Controller
/admin/*	auth → verified → role:admin	AdminExamController, AdminUserController, AdminFinanceController
/student/*	auth → verified → role:student	StudentExamController, StudentFinanceController
/evaluator/*	auth → verified → role:evaluator	EvaluatorScriptController, EvaluatorFinanceController
/payment/*	none (outside auth)	StudentFinanceController (callbacks only)

14. Security

Threat	Protection	Implementation
SQL Injection	Parameterized queries	Eloquent ORM uses PDO prepared statements
Weak Passwords	Bcrypt + strength rules	<code>CheckPasswordStrength.php</code>
XSS	Auto-escaping	Blade <code>{{ }}</code> escapes all output
CSRF	Token validation	<code>@csrf</code> in every form
Unauthorized Access	Double middleware	<code>auth</code> + <code>RoleMiddleware.php</code>
Payment Tampering	Server-to-server IPN	<code>StudentFinanceController::success()</code>
Session Hijacking	SameSite cookies	Callbacks outside auth group
Bad Uploads	Type + size validation	Only PDF, max 20MB
Double Spending	Immediate deduction	Balance deducted at request time

15. Conclusion

EvalFlow is a complete examination platform using Laravel 11, MySQL, and SSLCommerz that solves manual exam management problems:

- **Structured lifecycle:** Course → Exam → Questions → Timed Submissions → Per-question Grading → Results
- **Role isolation:** 3 dashboards with middleware-enforced access (`RoleMiddleware.php`)
- **Transparency:** Students see per-question marks via `script_marks` table
- **Integrated finance:** SSLCommerz deposits → auto fee deduction → evaluator earnings → admin-approved payouts
- **Data integrity:** FK cascades, UNIQUE constraints, DB transactions, server-side validation

The 8-table database handles edge cases: late submissions blocked, duplicates prevented, insufficient balances caught, re-grading supported via `updateOrCreate` , double-spending prevented by immediate balance deduction.